Folios — Data Handling

FOLIOS · FOLIOSHQ.COM

LAST UPDATED: 2026-06-19 (F6 — SEMANTIC RECALL: FOLIO_EMBEDDINGS, RLS SCOPE, EMBEDDINGS-PROVIDER BOUNDARY)

This document inventories what data Folios stores, where it lives, who can access it, and what crosses the boundary to third-party AI inference (Anthropic).

For broader security posture, see security.md.

Data inventory

User-supplied data (stored in Supabase Postgres)

Table	Contents	Sensitivity
`folio_accounts`	Account name, type, tier, address, lat/lng, account number, status, owner, parent account	User business data
`folio_contacts`	Name, role, email, phone, LinkedIn URL, leader/primary flags	Business contact info
`folio_meetings`	Meeting date, method, attendees, notes, AI summary, follow-up date, status	User-typed notes
`folio_items`	Action item text, assignee, due date, done flag	User business data
`folio_tasks`	Unified items+tasks home (Gauge V3 Phase 1; dual-written from Pip plan apply) — title, description, project link, assignee, due, custom fields	User business data
`folio_cadences`	Cadence label, frequency, day-of-week, meeting time	User schedule
`gauge_projects`	Project name, description, stages, custom fields	User business data
`folio_quick_tasks`	Task text, due date, completed flag	User business data
`folio_account_notes`	Per-account notes, plain text	User business data
`folio_account_updates`	Update calendar entries (catalog/pricing/integration changes)	User business data

Authentication data (stored by Supabase Auth, not directly readable by Folios)

Field	Stored by	Notes
Email	Supabase Auth	Used for sign-in and MFA
Password hash	Supabase Auth	bcrypt; never visible to Folios code
MFA secret	Supabase Auth	TOTP shared secret if MFA enabled
Session tokens	Supabase Auth	JWT; client and server side

Multi-tenant / org data

Table	Contents
`folio_orgs`	Org name, billing plan
`folio_org_members`	User membership in orgs, role, default lens
`folio_activity`	Audit log of writes (user_id, action, resource, timestamp)

Pip learning data (the "V2 brain")

Table	Contents	Purpose
`pip_assignment_hints`	Task pattern → assignee mapping per account	Pip learns who does what
`pip_correction_log`	User corrections to Pip output (rejected rows, edited text, reassignments)	Pip learns user preferences
`pip_correction_log_archive`	Compressed/older corrections	Long-term memory
`pip_glossary`	User-taught terms, aliases, definitions	Pip learns user vocabulary
`pip_account_state`	Per-account distilled "lessons learned"	Pip's running notebook
`pip_promise_log`	Tracking of Pip-suggested follow-ups and their outcomes	Calibration
`folio_embeddings`	Vector embeddings of the user's own notes/summaries (see below)	Semantic recall — Pip finds relevant past context by meaning, not just recency

SEMANTIC RECALL (`FOLIO_EMBEDDINGS`, PGVECTOR)

Folios embeds a copy of four kinds of the user's own content so Pip can answer "what did we decide about X months ago" by **meaning**:

source_type	Source column	Author
`meeting_notes`	`folio_meetings.notes` (verbatim)	user
`meeting_summary`	`folio_meetings.pip_summary`	Pip (already generalized — see data line)
`project_note`	`folio_meetings.project_notes` (verbatim)	user
`account_update`	`folio_account_updates.title` + `.description` (verbatim)	user

- **RLS scope:** `folio_embeddings` carries an RLS policy filtering on `auth.uid() = user_id`. The recall function (`match_folio_embeddings`) runs `SECURITY INVOKER` **and** adds an explicit `user_id = auth.uid()` predicate, and is only ever called with the caller's JWT — so a user can only ever retrieve their **own** embeddings, scoped to the account by default.
- **Data line:** the three user-authored sources are the user's own words (the same text already stored verbatim in their notebook). The one Pip-authored source (`pip_summary`) is already required to generalize any quantitative business data at generation time, so embedding introduces no new retention of company numbers. No revenue/volumes/rosters are embedded.
- **Embeddings provider:** chunk text is sent to the embeddings provider (default OpenAI `text-embedding-3-small`, server-side) to produce the vector; see "What crosses the boundary" below. The corpus is rebuilt only when the underlying note changes (per-source content fingerprint).

Observability data

Table	Contents	Retention
`folio_errors`	Client-side errors (React, network, Pip), with stack and context	No automatic deletion today (planned: 90 days)
`folio_pip_usage`	Per-call Pip API usage (tokens, mode, timestamp)	No automatic deletion today (planned: 90 days)

The corporate data line (what never enters Folios)

Folios is a personal notebook, deliberately separated from the user's employer's quantitative business data. Two enforced properties:

1. **Never solicited.** No AI surface in Folios asks for revenue figures, transaction volumes, customer counts, shop lists/rosters, pricing, or contract terms. Where business performance matters, questions are directional ("trending up or down?") — never numeric.
2. **Sanitized at the source.** The Email/Teams Digest Handoff — the one structured channel from the user's work environment into Folios — runs on a fixed prompt whose first rule excludes all quantitative business data. The digest carries account names, people's names, qualitative conclusions, and dates only. Folios parses it deterministically (no AI call) and the user reviews every row before anything is filed.

Raw user-typed notes are stored verbatim (it's the user's notebook), but AI-authored memory (facts, profiles, summaries, account state) is instructed to generalize any quantitative business data rather than retain it.

What crosses the boundary (Anthropic + the embeddings provider)

Pip is the only feature in Folios that sends user data outside the Supabase + Vercel boundary. Every other interaction is fully self-contained within the user's session and the user's database rows. Two external AI services are involved: **Anthropic** (Pip's language model) and the **embeddings provider** (semantic recall).

Embeddings provider (default OpenAI, server-side only): to build the semantic-recall corpus, the text of the user's own notes/summaries/updates (see `folio_embeddings` above) is sent over TLS to the embeddings provider, which returns a numeric vector. Only the user's own content is sent; no passwords, JWTs, MFA secrets, or other users' data. At query time, only the user's typed question is embedded. This is the same data-line-clean content already sent to Anthropic — it stays within the corporate data line (no revenue/volumes/rosters). Recall is **off** until `OPENAI_API_KEY` is configured; with no key, nothing is sent and Pip falls back to recency-only context.

When Pip runs, the following is sent to Anthropic via TLS:

Always sent:

- User-typed meeting notes (the draft Pip is summarizing or the question Pip is answering)
- System prompt (Pip's personality + instructions)

Sent contextually, based on the Pip mode:

- Account names, types, tiers, status, account context (Quick Notes)
- Contact names, roles, email
- Open items: text, assignee, due date, completion status
- Recent meetings: date, method, summary, follow-up date
- Active Gauge projects: name, status, stages
- Glossary entries: terms, aliases, definitions
- Org members: name, email (so Pip knows who could be assigned a task)
- Pip's own learning state: assignment hints, lessons learned, recent corrections (last 10)

- Semantic recall: a few of the user's own past notes/summaries that are most relevant to the question (retrieved from `folio_embeddings`, the user's own content, account-scoped)

Never sent:

- User passwords or password hashes (Supabase Auth holds these; Folios code never sees them)
- MFA secrets
- Session JWTs
- Other users' data (RLS scopes context to the requesting user; Pip only sees what the user could see)
- Anything from `folio_errors` (observability is local)

> ***Chat agent loop (read tools).*** *When Pip's chat loop fetches more on demand (a deep account read, open/stalled work, a notes search), the query runs against the* ***caller's own*** *RLS-scoped session — it can only read the requesting user's own notebook rows, the same data Pip could already be sent. The read tools* ***write nothing*** *and never solicit quantitative business data (they take account names, topics, and status filters — never figures), so they neither cross the corporate data line nor retain anything. See `ai-governance.md` → "Chat agent loop (bounded)".*

- Anything from `folio_pip_usage` (cost telemetry is local)
- Anything from `folio_activity` (audit log is local)

How Anthropic handles the data

- Calls go to Anthropic's standard API (claude-haiku-4-5-20251001 for Pip, occasionally Sonnet for high-value synthesis).

- Anthropic does **not train** on API customer inputs by default. Folios's calls are not used for model training.
- Anthropic retains API inputs and outputs for 30 days for abuse monitoring, then deletes them. This is governed by Anthropic's standard API terms.
- Anthropic is SOC 2 Type 2 attested.

The single bidirectional boundary is: `(Folios serverless function) ↔ (Anthropic API)`. No other AI vendor receives Folios data.

Access control

Row-Level Security

Every user-data table has RLS policies that filter on `auth.uid()`. The database physically refuses to return rows that don't belong to the requesting user. This isn't an application-level check — it's a Postgres-enforced rule, applied even if application code is buggy.

Example policy on `folio_accounts`:

```
create policy "accounts_select_own" on folio_accounts
  for select using (auth.uid() = user_id);
```

Same pattern applies to insert, update, and delete.

Org-shared data

For multi-tenant teams, RLS expands to org membership:

```
create policy "accounts_select_org" on folio_accounts
  for select using (
    user_id = auth.uid()
    OR org_id IN (select org_id from folio_org_members where
user_id = auth.uid())
  );
```

Only users who are members of the same org see each other's data.

Role differentiation

Within an org, roles (`owner | admin | member`) control admin capabilities:

- `owner` — full settings, billing, invite/revoke users
- `admin` — invite users, manage org-level settings
- `member` — read/write their assigned data

Service-role keys

Service-role keys (bypass RLS) are **never** used from client code. They exist only in server-side maintenance scripts (e.g., `scripts/seed-demo-data.js`) that the user runs locally with their own service-role credentials.

Data lifecycle

Creation

- All writes flow through Supabase via the JS client.
- Every write fires a `logActivity(...)` call into `folio_activity` for the audit trail.

Updates

- Same RLS rules apply to updates.
- Realtime subscriptions notify other open sessions of the same user (multi-device sync).

Deletion

- **Hard deletes** are reserved for explicit user-initiated actions (e.g., deleting a meeting note).
- **Soft deletes / inactive flag** is the default for accounts and users — preserves history but excludes from active surfaces.
- **Account merge** (post-acquisition workflow) re-parents all child rows from the source account to the target, then marks the source inactive. The `folio_merge_accounts` Postgres function atomically re-parents: meetings (`folio_meetings`), contacts (`folio_contacts`), tasks (`folio_tasks`), items (`folio_items`), updates (`folio_account_updates`), Gauge projects (`gauge_projects`), cadences (`folio_cadences`), snapshots (`folio_account_snapshots`), correction log (`pip_correction_log`), assignment hints (`pip_assignment_hints`), promise log (`pip_promise_log`), the account reference embedded in drip-question suggestions (`folio_pip_questions.suggestion` jsonb — both `account_id` and the display `account_name`), and array columns (`folio_meetings.account_ids` , `folio_cadences.account_ids`) via `array_replace` . Contact aliases (`folio_contact_aliases`) need no re-parenting — they key on `contact_id` , and contact rows are themselves re-parented, so aliases follow their contact automatically. After re-parenting, the source account's `is_inactive` flag is set and `merged_into_account_id` is recorded.
- **User deletion** triggers a cascade through all `folio_*` tables via the foreign-key `on delete cascade` rules.

Export

- Per-account JSON export available (Account → Export).
- Bulk-user export not yet built (planned).

Retention

- **User-controlled data** (accounts, meetings, items, etc.): retained indefinitely until the user deletes it.
- `folio_errors`: planned 90-day rolling deletion (not yet enforced — single-user pilot has low volume).
- `folio_pip_usage`: planned 90-day rolling deletion (not yet enforced).
- `pip_correction_log`: compressed and archived to `pip_correction_log_archive` every ~5 meetings per account; compressed lessons distilled into `pip_account_state.lessons_learned`.

Geographic residency

Service	Region
Supabase Postgres	US (us-east-1 region)
Vercel hosting	US (global edge, but origin US)
Anthropic API	US

No data leaves the US under current configuration.

Data subject rights (GDPR/CCPA posture)

Folios stores no EU personal data today (single-user pilot, US-based user). For future EU/CCPA exposure, the following user rights are already supported by the architecture:

Right	How it's supported
Access	Per-account JSON export
Rectification	User can edit any field directly
Erasure	Account deletion cascades through all tables
Portability	JSON export is the portability mechanism
Restriction	Inactive flag suppresses processing without deletion
Objection	User can disable Pip globally (Pip is opt-out per
session via toggle; planned hardline-opt-out toggle in Settings)	
Audit	Activity log per user

A formal DPA, ROPA, and DPIA would be drafted as part of multi-user or EU expansion.

Backups

Managed by Supabase:

- Daily automated backups (managed service)
- Point-in-time recovery (PITR) on Pro tier with RPO < 5 minutes

Folios does not maintain a separate backup of user data; reliance on Supabase's managed backup is the documented strategy.

Contact

For data-handling questions or data-subject requests: chris.vasconcellos97@gmail.com